

Programmazione ad oggetti

Corso di Laurea in Ingegneria Informatica, Elettronica e delle Telecomunicazioni

Prova di programmazione

15 gennaio 2025

Si vuole implementare un buffer che permette di accumulare un numero variabile di stringhe per poi poterle scrivere tutte insieme in un `std::ostream`, come ad esempio `cout`.

Si chiede di:

1) Creare la classe `StringBuffer` definita qui sotto come classe astratta pura, e creare una classe `StringBufferImpl` che estende e implementa `StringBuffer`.

StringBuffer
<pre>/** Appends a new string. Returns this StringBuffer. */ virtual StringBuffer& append(const std::string& s); /** Appends a new char. Returns this StringBuffer. */ virtual StringBuffer& append(char c); /** Appends a new integer. Returns this StringBuffer. */ virtual StringBuffer& append(int n); /** Returns the total string length. */ virtual unsigned size() const; /** Prints the resulting string to an output stream. */ virtual void print(std::ostream& os) const; /** Empties the StringBuffer. */ virtual void clear();</pre>

Per l'implementazione della classe concreta `StringBufferImpl` si utilizzi internamente un array di `std::string`, oppure in alternativa una classe delle STL.

Provare le classi `StringBuffer` e `StringBufferImpl` col seguente codice (`test.cpp`):

```
StringBufferImpl sb;
testBuffer(sb);
sb.print(cout);
sb.append('1').append(234);
sb.print(cout);
cout << "Lunghezza totale: " << sb.size() << endl;
```

dove `testBuffer()` è la seguente funzione:

```
void testBuffer(StringBuffer& sb) {
    string text[] = {"Questo ", "e' ", "un ", "testo ", "di ", "esempio."};
    for (auto& s : text) sb.append(s);
}
```

Questo programma di test dovrebbe dare in uscita un output del tipo:

```
Questo e' un testo di esempio.  
Questo e' un testo di esempio.1234  
Lunghezza totale: 34
```

2) Sovraccaricare l'operatore `operator<<` in modo da poter scrivere uno `StringBuffer` su `std::ostream`. Provare in nuovo operatore aggiungendo la seguente riga al programma di test:

```
cout << sb << endl;
```

3) Aggiungere la funzione per leggere uno `StringBuffer` da file:

```
void read(const string& file_name, StringBuffer& sb);
```

e provarlo aggiungendo al programma di test il seguente codice:

```
sb.clear();  
string file_name= "esempio.txt";  
read(file_name, sb);  
sb.print(cout);
```

Come file di testo "esempio.txt" utilizzare:

```
Lorem ipsum dolor sit amet,_  
consectetur adipiscing elit,_  
sed do eiusmod_  
tempor incididunt_  
ut labore et dolore magna aliqua.
```

4) Aggiungere la seguente funzione che legge uno `StringBuffer` da file e lo restituisce come `unique_ptr< StringBuffer>`:

```
unique_ptr<StringBuffer> read(const string& file_name);
```

e provarlo aggiungendo le seguenti righe di codice:

```
auto sb_ptr= read(file_name);  
sb_ptr->print(cout);
```